AI CODING PROMPT PACK - PHASE 2

FLS Checker - Real Analysis Engine

From a working demo shell to a real DXF-parsing, code-checking engine - copy-paste-ready prompts

Version 1.0 - Use after Phase 1 (Tasks 1-6) is complete
August 2026

1. Where You Are Now

Phase 1 (Tasks 1-6) made the demo honest and dynamic: real upload, real processing status, real per-drawing data, no hardcoded drawing ID. What still isn't real: the actual analysis. Today, an uploaded file still gets the same seeded example findings copied onto it - nothing yet reads the file's real geometry or checks it against real code clauses.

> ***Phase 2's single goal: replace 'copy the seeded example findings' with 'actually parse the file and calculate real findings from real code data.' Everything else - the UI, the review workflow, the export - already works and should NOT change in this phase.***

You now have the real code data ready to use: uae_fls_code_clauses_business_occupancy.json, sourced directly from your UAE Fire and Life Safety Code PDF (Chapter 3, pages 235-350). That file is the input to Task 1 below.

2. How to Use This Document

- Paste Section 3 (Updated Context Prompt) first, in a new session, along with the JSON file attached or referenced.
- Then paste Task 1 through Task 7 from Section 4, one at a time, in order.
- After each task, test it yourself against the 5-floor Dubai test PDF from earlier, not just the original seeded example - that's the only way to know real parsing is actually working on a new file, not just replaying memorized results.
- Keep repeating the Section 5 guardrails from the Phase 1 pack - they still apply.

3. Updated Context Prompt (paste this first, new session)

```
Continuing work on FLS Checker. Phase 1 is complete: file upload is
wired to the backend, processing status is polled, the review
screen uses a dynamic drawing ID (no more hardcoded
drawing-al-noor-l06), the real floor plan renders from extracted
element data, and loading/error/empty states exist.

WHAT'S STILL FAKE (this is what Phase 2 fixes):
When a file is uploaded, 'demo processing' still just copies the
original seeded example's findings onto the new file's record. No
code yet actually reads the uploaded DXF/PDF's real geometry or
checks it against real fire-safety code rules.

WHAT I NOW HAVE THAT I DIDN'T BEFORE:
A file called uae_fls_code_clauses_business_occupancy.json -
real UAE Fire and Life Safety Code clause data (occupant load
factors, exit count thresholds, travel distance limits, dead-end
limits, stair/corridor width minimums), sourced from the actual
code PDF, Chapter 3 'Means of Egress'. This replaces any
placeholder/hardcoded numbers currently in the rules logic.

THE GOAL OF THIS PHASE:
Replace the demo-processing placeholder with a real pipeline:
parse the file's actual geometry -> calculate real travel
distances and occupant loads -> compare to the real code clause
```

```
data -> generate real, correctly-cited findings.

RULES THAT STILL APPLY:
- Don't touch the review workflow, export, or UI unless a task
  asks you to - they already work.
- Explain changes in plain English, tell me how to test each
  one myself, and ask me if anything is ambiguous rather than
  guessing.
- One task at a time - confirm each works before the next.

Confirm you understand, then wait for Task 1.
```

4. Phase 2 Tasks (paste one at a time, in order)

TASK 1 of 7 - Load real code clause data into the database

```
TASK: Load the attached uae_fls_code_clauses_business_occupancy.json
into a proper CodeClause table, replacing any hardcoded or
placeholder clause numbers currently used anywhere in the app.

Specifically:
1. Create a CodeClause table/model if one doesn't already exist,
   with fields matching the JSON: clause_id, topic, occupancy,
   requirement_type, value, unit, condition (if present),
   source_table, source_page.
2. Write a one-time script or seed function that loads all
   entries from the JSON file into this table.
3. Find every place in the current code where a code limit is
   hardcoded (e.g. a fixed travel distance number) and make a
   list for me of what you found - don't change the logic yet,
   just report it.

DEFINITION OF DONE:
- Querying the database shows all clauses from the JSON file
  as real rows.
- I have a list of every hardcoded limit currently in the code,
  so we know what Task 5 needs to replace.
- Explain what you did and how to verify the data loaded
  correctly.
```

TASK 2 of 7 - Real DXF parsing (walls, doors, rooms, exits)

```
TASK: Replace demo processing with real DXF parsing using the
ezdxf library. When a .dxf file is uploaded, extract real wall,
door, room, and exit geometry from it instead of copying the
seeded example.

Specifically:
1. Add ezdxf as a dependency if not already present.
2. On DXF upload, parse the file's layers/entities to identify
   walls (lines/polylines), doors (blocks or gaps), and room
   boundaries (closed polylines or labeled areas).
3. Store the extracted geometry using the existing
   ExtractedElement structure (or equivalent) so the current
   floor plan viewer can render it without changes.
4. If parsing fails or finds nothing usable, show a clear error
   instead of silently falling back to demo data.
5. Keep the seeded example drawing working exactly as before -
   don't reprocess it through this new pipeline unless I ask.

DEFINITION OF DONE:
- Uploading a real DXF file (use the Dubai test floor plan set)
  produces extracted elements that are genuinely different from
  the seeded example - actual walls/doors/rooms from THAT file.
- The existing plan viewer correctly displays this new real
```

```
  geometry.
- Explain what you changed and how to test it with a specific
  file.
```

TASK 3 of 7 - Build the walkable path graph and calculate travel distance

```
TASK: Using the real extracted geometry from Task 2, build a
walkable path graph and calculate actual travel distance from
each room to the nearest exit.

Specifically:
1. Add NetworkX and Shapely as dependencies.
2. From the extracted rooms/corridors/exits, build a graph where
   rooms and corridor segments are nodes and physical
   connections (doorways, openings) are edges.
3. Identify which extracted elements are exits (vs. regular
   interior doors) - if this can't be determined automatically
   from the DXF layer/label data, tell me what manual tagging
   approach you'd suggest instead of guessing.
4. Calculate the shortest walkable path distance from each
   room's centroid to the nearest exit, using the drawing's
   real scale (not an assumed one).
5. Store this calculated distance against each room.

DEFINITION OF DONE:
- For the Dubai test floor plan, the calculated travel
  distances are real numbers derived from that file's actual
  geometry and scale - not copied from anywhere else.
- You can show me the calculated distance for at least 3
  different rooms and explain how each was derived.
- Explain what you changed and how to verify a distance
  calculation by hand for one simple room as a sanity check.
```

TASK 4 of 7 - Calculate real occupant load per room

```
TASK: Calculate real occupant load for each extracted room,
using its actual area and the occupant load factor from the
CodeClause data loaded in Task 1.

Specifically:
1. Calculate each room's real floor area from its extracted
   boundary geometry (Shapely).
2. Look up the applicable occupant load factor from the
   CodeClause table (e.g. 9.3 m2/person for regular office, per
   UAE-FLS-3.13-BUS-REG) rather than a hardcoded number.
3. Calculate occupant load = room area / occupant load factor,
   and store it against each room.
4. If a room's occupancy type isn't specified anywhere yet,
   default to 'Regular office areas' and flag this assumption
   clearly rather than silently guessing a different type.

DEFINITION OF DONE:
- Every extracted room has a calculated occupant load based on
  its real area and the real code factor.
```

```
- I can spot-check one room's math myself (area divided by 9.3)
  and get the same number the app shows.
- Explain what you changed and how to test it.
```

TASK 5 of 7 - Real rules check: generate real, code-cited violations

```
TASK: Replace the demo-processing placeholder findings entirely.
Compare the real calculated values from Tasks 3-4 against the
real CodeClause thresholds from Task 1, and generate real
Violation records - only for rooms that actually exceed a limit.

Specifically:
1. For each room, compare its calculated travel distance against
   the applicable travel distance clause (remember: sprinklered
   vs non-sprinklered gives very different limits - see Task 6).
2. Compare each room/floor's occupant load against the required
   number-of-exits clauses, and flag if there aren't enough
   exits for that load.
3. Every generated Violation must store the real clause_id it
   references (from the CodeClause table), the actual measured
   value, and the actual limit value - no more generic/example
   text.
4. Remove or clearly disable the old demo-processing code path
   that copied the seeded example's findings, so it can't
   accidentally run instead of the real check.

DEFINITION OF DONE:
- Uploading the Dubai test floor plan produces violations that
  are genuinely calculated from that file - different rooms,
  different numbers than the original seeded example.
- Each violation clearly cites a real clause_id and shows real
  measured-vs-limit values.
- The original seeded demo drawing still works (for comparison/
  fallback purposes) but the new upload path no longer touches
  demo data at all.
- Explain what you changed and how to verify a violation is
  correct by checking it against the CodeClause reference PDF.
```

TASK 6 of 7 - Add project-level inputs: occupancy type and sprinkler status

```
TASK: Add two pieces of information the app cannot determine
from a drawing alone, but which are required for Task 5's
checks to pick the correct limits: occupancy type and whether
the building is sprinklered.

Specifically:
1. Add fields to the project or drawing creation form: occupancy
   type (default 'Business - Regular office areas' for now) and
   sprinklered (yes/no).
2. Make Task 5's rules check use these values to select the
   correct CodeClause rows (e.g. the sprinklered travel distance
   clause vs. the non-sprinklered one).
3. If these fields aren't filled in, don't guess silently -
   require them before processing can run, with a clear message
```

```
   explaining why they're needed.

DEFINITION OF DONE:
- Creating a project asks for occupancy type and sprinkler
  status.
- Changing sprinkler status on the same drawing changes which
  violations are flagged (since the limits are very different),
  proving the app is actually using this input, not ignoring it.
- Explain what you changed and how to test both settings.
```

TASK 7 of 7 - Validate against the test floor plan set

```
TASK: Run the full real pipeline (Tasks 2-6) against all 5 floors
of the Dubai test floor plan PDF, and produce a validation
summary so I can sanity-check the results myself before trusting
them.

Specifically:
1. Process each of the 5 floor plans through the real pipeline.
2. For each floor, produce a simple summary: how many rooms were
   extracted, how many violations were found, and for each
   violation, the room name, measured value, limit value, and
   clause_id.
3. Flag anything that looks clearly wrong (e.g. a room with zero
   area, an occupant load of zero, a travel distance of zero) -
   these usually mean extraction failed silently rather than the
   room genuinely having no issues.
4. Don't fix issues you find yet - just report them clearly so I
   can decide what to prioritize next.

DEFINITION OF DONE:
- I have a clear, readable summary of what the real pipeline
  found on all 5 real test floors.
- Any suspicious/likely-broken results are clearly flagged, not
  silently included as if they were trustworthy.
- Explain what you found and what you'd recommend fixing first.
```

5. After Phase 2

Once these 7 tasks are done, your app will do the actual thing the whole product is for: read a real drawing and produce real, code-cited findings - with no LLM involved anywhere yet, exactly as planned. What's left after this, in order, is: expand rule coverage beyond travel distance and exit count (fire-rating, corridor width enforcement, dead-end checks), add a second occupancy type or jurisdiction, and only then - once this deterministic core is proven - add the RAG/LLM explanation layer from the architecture document. Come back for that prompt pack once Phase 2 is solid and tested.

6. Quick Reference - What Changed From Phase 1

Area	Phase 1 result	Phase 2 result
Findings shown	Real UI, but copied from one seeded example	Calculated from the actual uploaded file's real geometry
Code limits used	Hardcoded/placeholder numbers	Real values from uae_fls_code_clauses_business_occupancy.json
Floor plan analysis	None - no code reads the file's content	Real DXF parsing (ezdxf) extracts actual geometry
Travel distance	N/A	Calculated via a real path graph (NetworkX) and real scale
Occupant load	N/A	Calculated from real room area / real code factor
Sprinkler/occupancy info	N/A	Captured as project-level input, required before processing